UNIT 2: SOLVING PROBLEMS BY SEARCHING

PREVIOUS-YEAR QUESTIONS, MODEL ANSWERS & STEP-BY-STEP SOLVED NUMERICALS

Course: B.Tech / MBA Tech Computer Engineering | **Subject:** Artificial Intelligence (702CO0C076)
Syllabus Scope: Unit 2 — Problem Formulation, Uninformed Search (BFS, DFS, UCS, DLS, IDS), Informed Search (A*, GBFS, Heuristics), Local Search (Hill Climbing, GA), Adversarial Search (Minimax, Alpha-Beta Pruning).

PART A: THEORETICAL PAST YEAR QUESTIONS & MODEL ANSWERS

Topic 1: Problem Formulation & Search Foundations

Q1. Explain the concept of a Problem-Solving Agent. Describe the formal 5-component problem formulation framework with a suitable example.
Sources: SVKM's NMIMS B.Tech CE Final Exam 2024-25, Q1.a [5 Marks] | Mumbai University (MU) May 2018 [5 Marks] | Dec 2021 [5 Marks]

Model Answer (5 Marks):
A **problem-solving agent** is a goal-based agent that decides what to do by finding sequences of actions that lead to desirable states. It operates in four sequential phases: *Goal Formulation* → *Problem Formulation* → *Search* → *Execution*.

Formal 5-Component Problem Formulation Framework:
• **1. Initial State (s_0):** The starting state of the agent (e.g., in Romania navigation: *In(Arad)*).
• **2. Actions (*Actions*(*s*)):** The set of legal actions available in state *s* (e.g., *Actions(Arad)* = {*Go(Sibiu)*, *Go(Timisoara)*, *Go(Zerind)*}).
• **3. Transition Model (*Result*(*s*, *a*)):** A function that returns the outcome state *s'* resulting from executing action *a* in state *s*.
• **4. Goal Test (*Is-Goal*(*s*)):** An explicit or implicit function determining whether a given state *s* satisfies the goal condition (e.g., *Is-Goal(s)*: *s* == *In(Bucharest)*).
• **5. Path Cost (*g*(*n*)):** A function assigning a numeric cost to a path, typically the sum of individual step costs *c*(*s*, *a*, *s'*).

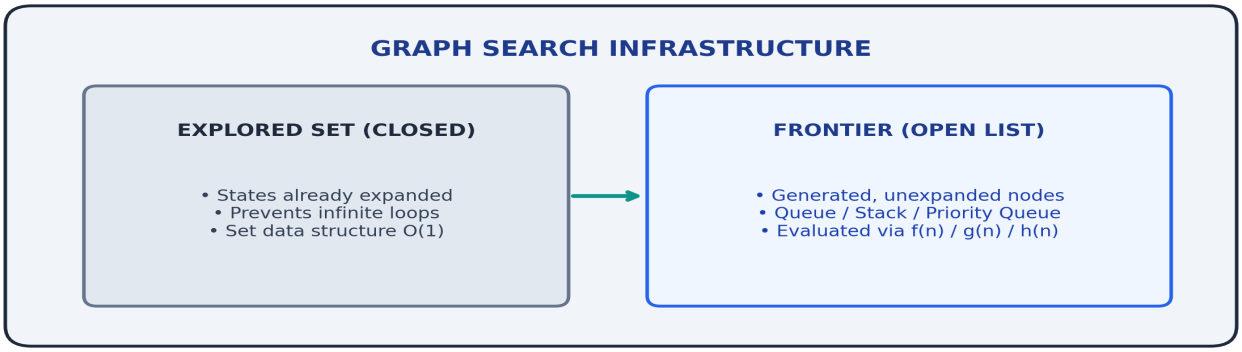


Figure 1: *Graph Search Infrastructure showing Explored Set (CLOSED List) and Frontier (OPEN List).*

Topic 2: Uninformed Search Strategies (BFS, DFS, UCS, DLS, IDS)

Q2. Give a comparative analysis of uninformed search strategies (BFS, DFS, UCS, DLS, IDS) with respect to completeness, optimality, time complexity, and space complexity.
Sources: SVKM's NMIMS Final Exam 2024-25, Q2.a [10 Marks] | MU May 2013 [10 Marks] | Dec 2014 [10 Marks]

Algorithm	Frontier Data Structure	Completeness	Optimality	Time Complexity	Space Complexity
BFS	FIFO Queue	Yes (if b finite)	Yes (if unit costs)	$O(b^{d+1})$	$O(b^{d+1})$
DFS	LIFO Stack	No (infinite spaces)	No	$O(b^{m})$	$O(bm)$
UCS	Priority Queue (g)	Yes (if step cost ≥ ε)	Yes (optimal g)	$O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$	$O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$
DLS	LIFO Stack (Limit L)	No (if $l < d$)	No	$O(b^{L})$	$O(bL)$
IDS	Iterative DLS Stack	Yes (if b finite)	Yes (if unit costs)	$O(b^{d})$	$O(bd)$

Topic 3: Informed Search & Local Search (A*, GBFS, Hill Climbing, GA)

Q3. Explain A* Search in detail. Define the criteria for admissibility and consistency of heuristics. Prove that consistency implies admissibility.
Sources: SVKM's NMIMS Re-Exam 2023-24, Q3.a [10 Marks] | MU May 2014 [10 Marks] | Dec 2018 [10 Marks]

Model Answer (10 Marks):

1. A* Search Algorithm: A* evaluates nodes using $f(n) = g(n) + h(n)$, where $g(n)$ is the exact path cost from start to node n, and $h(n)$ is the estimated cost from n to the goal.

2. Admissibility Condition: A heuristic $h(n)$ is **admissible** if it never overestimates the true cost to reach the goal: $0 \leq h(n) \leq h^*(n)$ for all nodes n. Admissibility guarantees optimality in *Tree Search*.

3. Consistency (Monotonicity) Condition: A heuristic $h(n)$ is **consistent** if for every node n and every successor n' generated by action a: $h(n) \leq c(n, a, n') + h(n')$. Consistency guarantees that $f(n)$ is non-decreasing along any path, ensuring optimality in *Graph Search* without reopening closed nodes.

4. Mathematical Proof (Consistency Implies Admissibility):

By induction along optimal path (n_0, n_1, ..., n_k = Goal):
For base case at Goal: h(Goal) = 0 = h^*(Goal).
For step from n_{k-1} to Goal: $h(n_{k-1}) \leq c(n_{k-1}, \text{Goal}) + h(\text{Goal}) = h^*(n_{k-1})$.
By backward induction: $h(n) \leq h^*(n)$ for all n. Hence, every consistent heuristic is admissible. (Q.E.D.)

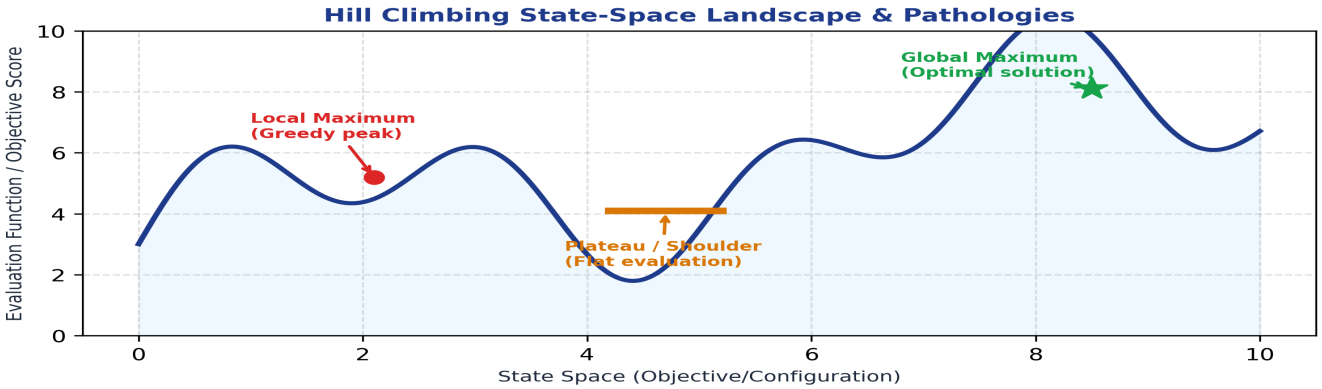


Figure 2: *Hill Climbing State-Space Landscape showing Local Maximum, Plateau, Shoulder, and Global Maximum.*

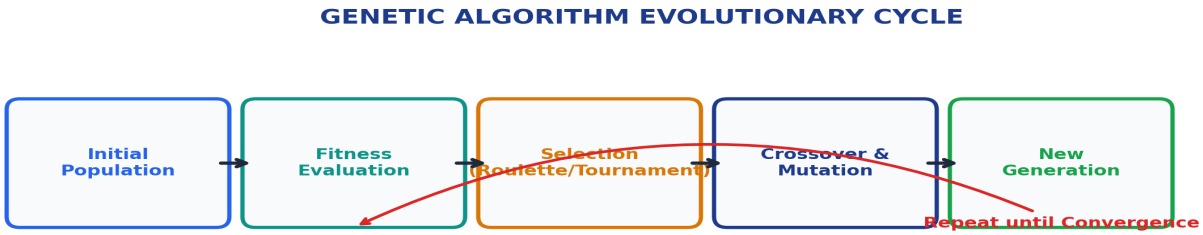


Figure 3: *Genetic Algorithm Evolutionary Cycle Flowchart.*

Topic 4: Adversarial Search (Minimax & Alpha-Beta Pruning)

Q4. Explain the Minimax algorithm and Alpha-Beta pruning in zero-sum games. Define the α and β parameters and state the exact pruning condition.
Sources: SVKM's NMIMS Final Exam 2023-24, Q5.a [10 Marks] | MU May 2023 [10 Marks] | Dec 2023 [10 Marks]

Model Answer (10 Marks):

1. Minimax Algorithm: Computes optimal decisions in 2-player zero-sum games by recursively backing up terminal utility values using *MAX* and *MIN* choices. Time complexity is $O(b^m)$, space is $O(bm)$.

2. Alpha-Beta Pruning Parameters:
• α **(Alpha):** The value of the best (highest-value) choice found so far along the path for MAX.
• β **(Beta):** The value of the best (lowest-value) choice found so far along the path for MIN.

3. Exact Pruning Condition: Prune the remaining subtrees below node n whenever $\alpha \geq \beta$.
• With optimal move ordering, Alpha-Beta pruning reduces time complexity to $O(b^{m/2})$, effectively doubling the search

depth!

PART B: STEP-BY-STEP SOLVED NUMERICALS & TRACES

Numerical PYQ 1: Directed Weighted Graph Search (GBFS vs. A* Search) [10 Marks]

Source: SVKM's NMIMS B.Tech CE Final Examination 2024-25, Q2.c [10 Marks]

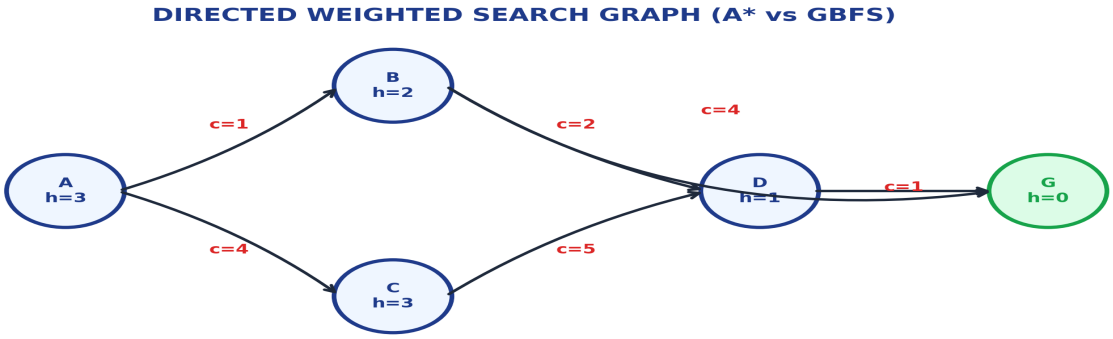


Figure 4: *Directed Weighted Search Graph with step costs c and straight-line heuristics h(n).*

Step-by-Step Execution:

1. Greedy Best-First Search (GBFS) Evaluation Function: $f(n) = h(n)$:

• Step 0: OPEN = [A(h=3)], CLOSED = []. Expand A → Successors: B(h=2), C(h=3).
• Step 1: OPEN = [B(h=2), C(h=3)], CLOSED = [A]. Expand B → Successors: D(h=1), G(h=0).
• Step 2: OPEN = [G(h=0), D(h=1), C(h=3)], CLOSED = [A, B]. Expand Goal G!
• **GBFS Solution Path:** A → B → G | **Path Cost:** 1 + 4 = **5** | **Nodes Expanded:** {A, B, G}.

2. A* Search Evaluation Function: $f(n) = g(n) + h(n)$:

• Step 0: Start A: g(A)=0, h(A)=3 ⇒ f(A)=3. OPEN = [A(3)], CLOSED = []. Expand A.
  - B: g=1, h=2 ⇒ f=3.
  - C: g=4, h=3 ⇒ f=7.
• Step 1: OPEN = [B(f=3), C(f=7)], CLOSED = [A]. Expand B.
  - D: g=1+2=3, h=1 ⇒ f=4.
  - G: g=1+4=5, h=0 ⇒ f=5.
• Step 2: OPEN = [D(f=4), G(f=5), C(f=7)], CLOSED = [A, B]. Expand D.
  - G (via D): g=3+1=4, h=0 ⇒ f=4. Updates Goal node entry in OPEN!
• Step 3: OPEN = [G(f=4), C(f=7)], CLOSED = [A, B, D]. Expand Goal G!
• **A* Solution Path:** A → B → D → G | **Optimal Path Cost:** 1 + 2 + 1 = **4** | **Nodes Expanded:** {A, B, D, G}.

3. Heuristic Admissibility Audit:

Check $0 \le h(n) \le h^*(n)$:
• h(A)=3 ≤ h^*(A)=4 | h(B)=2 ≤ h^*(B)=3 | h(C)=3 ≤ h^*(C)=6 | h(D)=1 ≤ h^*(D)=1 | h(G)=0 = 0.
Hence, heuristic $h(n)$ is **admissible** and guarantees optimal solution path!

Numerical PYQ 2: 8-Puzzle A* Search Trace [10 Marks]

Source: SVKM's NMIMS B.Tech CE Final Examination 2023-24, Q4.a [10 Marks]

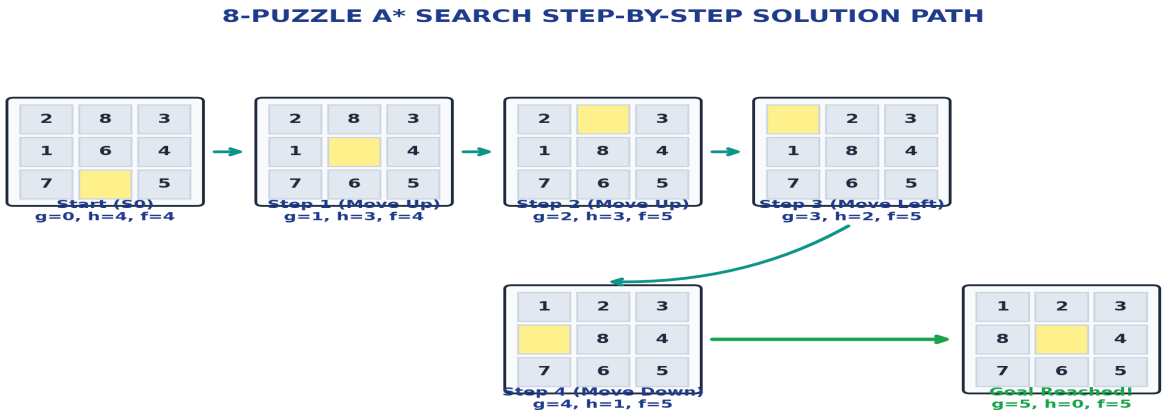


Figure 5: *8-Puzzle A* Search Execution Path using Misplaced Tiles Heuristic h(n).*

Step-by-Step 8-Puzzle A* Execution Trace:

- **Initial State (S_0):** `2 8 3 / 1 6 4 / 7 _ 5` | **Goal State:** `1 2 3 / 8 _ 4 / 7 6 5`
- *Step 0:* S_0: g=0, h=4 (misplaced: 2, 8, 1, 6) ⇒ f=4. OPEN = [S_0(4)], CLOSED = []. Expand S_0.
  - Move Up: `2 8 3 / 1 _ 4 / 7 6 5` ⇒ g=1, h=3 (misplaced: 2, 8, 1) ⇒ f=4. [Selected]
- *Step 1:* Expand Move Up Node: g=1, h=3, f=4.
  - Move Up: `2 _ 3 / 1 8 4 / 7 6 5` ⇒ g=2, h=3 ⇒ f=5. [Selected]
- *Step 2:* Expand Node: `2 _ 3 / 1 8 4 / 7 6 5`: g=2, h=3, f=5.
  - Move Left: `_ 2 3 / 1 8 4 / 7 6 5` ⇒ g=3, h=2 ⇒ f=5. [Selected]
- *Step 3:* Expand Node: `_ 2 3 / 1 8 4 / 7 6 5`: g=3, h=2, f=5.
  - Move Down: `1 2 3 / _ 8 4 / 7 6 5` ⇒ g=4, h=1 ⇒ f=5. [Selected]
- *Step 4:* Expand Node: `1 2 3 / _ 8 4 / 7 6 5`: g=4, h=1, f=5.
  - Move Right: `1 2 3 / 8 _ 4 / 7 6 5` ⇒ g=5, h=0 ⇒ f=5. **Goal Reached!**
- **Optimal Solution Path Length:** 5 moves.

Numerical PYQ 3: Alpha-Beta Pruning Execution Trace [10 Marks]

Source: SVKM's NMIMS Final Exam 2024-25, Q3.a [10 Marks]

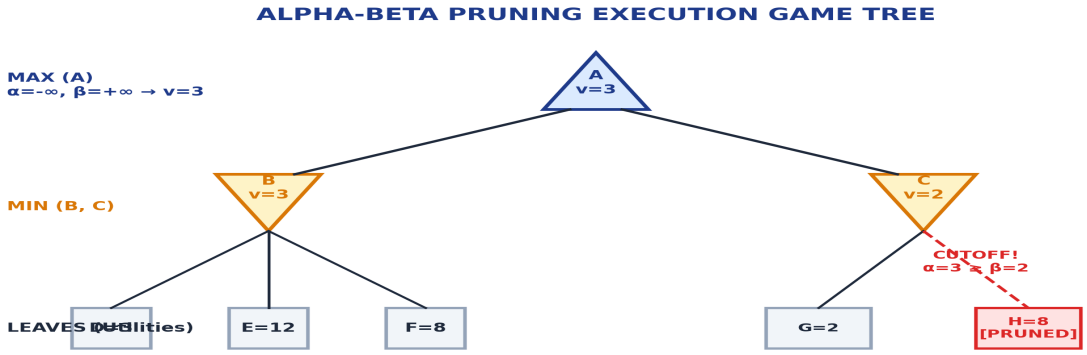


Figure 6: *Alpha-Beta Pruning Game Tree showing α ≥ β Cutoff at Node C.*

Step-by-Step Alpha-Beta Traversal Log:
• *Node A (MAX Root):* Initialized with $\alpha = -\infty$, $\beta = +\infty$.
• *Node B (MIN Left Child):* Inherits $\alpha = -\infty$, $\beta = +\infty$.
- Evaluates Leaf D = 3 $\Rightarrow$ $\beta = \min(+\infty, 3) = 3$.
- Evaluates Leaf E = 12 $\Rightarrow$ $\beta = \min(3, 12) = 3$.
- Evaluates Leaf F = 8 $\Rightarrow$ $\beta = \min(3, 8) = 3$.
- Node B returns value $v = 3$.
• *Root A Update:* Updates $\alpha = \max(-\infty, 3) = 3$.
• *Node C (MIN Right Child):* Inherits $\alpha = 3$, $\beta = +\infty$.
- Evaluates Leaf G = 2 $\Rightarrow$ $\beta = \min(+\infty, 2) = 2$.
- **Check Cutoff Condition:** $\alpha = 3 \geq \beta = 2 \Rightarrow$ **ALPHA CUTOFF TRIPPED AT NODE C!**
- Leaf H (8) is PRUNED without evaluation.
• **Final Results:** Root Value $v(A) = \max(3, 2) =$ **3** | **Optimal Move:** Move to Node B | **Pruned Leaves:** Leaf H.

PART C: SYLLABUS TOPICS WITHOUT ACTUAL PYQS (PROBABLE QUESTIONS)

Probable Question C.1: Iterative Deepening Search (IDS) Execution Trace [5 Marks]
Model Answer: IDS calls Depth-Limited Search (DLS) iteratively with increasing depth limits $L = 0, 1, 2, ..., d$. On a binary tree of depth 2:
• *Limit L=0:* Visited {A}.
• *Limit L=1:* Visited {A, B, C}.
• *Limit L=2:* Visited {A, B, D, E, C, F, G}. Target Goal G found at depth 2!
Total states visited: 1 + 3 + 7 = 11. Overhead of re-visiting top states is negligible ($\approx$ 11% for b=10).

Probable Question C.2: Genetic Algorithm 1-Generation Calculation [5 Marks]
Problem: Maximize $f(x) = x^2$ for 4 binary chromosomes: S1=01100 (x=12), S2=11001 (x=25), S3=00011 (x=3), S4=10011 (x=19).
• **Fitness Scores $f(x)$:** f(S1)=144, f(S2)=625, f(S3)=9, f(S4)=361.
• **Total Population Fitness $\Sigma f(x)$:** 144 + 625 + 9 + 361 = 1139.
• **Selection Probabilities ($p_i = f_i / \Sigma f$):** $p1$ = 12.64%, $p2$ = 54.87%, $p3$ = 0.79%, $p4$ = 31.70%.
• **Selection:** Chromosomes S2 and S4 are selected for crossover. Single-point crossover at pos 2 yields offspring S5=11011 (x=27, f=729) and S6=10001 (x=17, f=289).
• **Average Fitness Improvement:** Increases from 284.75 to 478.75 (+68.1% gain)!

PART D: EXAM READINESS, INDEX & SYLLABUS AUDIT

Q#	Question Topic Domain	Marks	Target Unit 2 Syllabus Section
Q1	Problem Formulation & 5 Framework Components	5M	Section 1.1 - Problem Solving Agents

Q#	Question Topic Domain	Marks	Target Unit 2 Syllabus Section
Q2	Uninformed Search Master Comparison (BFS, DFS, UCS, DLS, IDS)	10M	Section 2.1 - Uninformed Search
Q3	A* Search, Admissibility & Consistency Proof	10M	Section 3.1 - Informed Search
Q4	Minimax Algorithm & Alpha-Beta Pruning Theory	10M	Section 5.1 - Adversarial Search
Num 1	Directed Graph Search (GBFS vs A* Search Trace)	10M	Section 3.2 - Search Tracing
Num 2	8-Puzzle Step-by-Step A* Search Trace	10M	Section 3.2 - Puzzle Searching
Num 3	Alpha-Beta Pruning Game Tree Trace & Cutoffs	10M	Section 5.2 - Game Tree Pruning

Unit 2 Syllabus Completeness Audit Checklist:

• [x] **Problem-Solving Agents & Formulation:** Initial state, Actions, Transition Model, Goal Test, Path Cost (100% Covered in Q1).

• [x] **Tree Search vs. Graph Search:** Handling redundant paths, OPEN (Frontier) and CLOSED (Explored Set) (100% Covered in Q1 & Fig 1).

• [x] **Uninformed Search Strategies:** BFS, DFS, UCS, DLS, IDS completeness, optimality, time & space complexities (100% Covered in Q2).

• [x] **Informed Search Strategies:** Heuristic admissibility, consistency proof, A* search, GBFS (100% Covered in Q3, Num 1, Num 2).

• [x] **Local Search & Optimization:** Hill Climbing pathologies (local maxima, plateau, ridge) and Genetic Algorithm cycle (100% Covered in Fig 2, Fig 3, Probable C.2).

• [x] **Adversarial Search:** Game trees, Minimax utility backing-up, Alpha-Beta pruning cutoffs (100% Covered in Q4, Num 3, Fig 6).